

Reviewer Pack — Communication Complexity Lower Bound for AC^0 PARITY via Real...

Entry b77a7e4e1eda · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 03:40:45 UTC

Communication Complexity Lower Bound for AC^0 PARITY via Real Algebraic Geometry

Entry ID: b77a7e4e1eda **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 03:40:45 UTC

1. Conjecture

Field A (mathematical branch): COMMUNICATION_COMPLEXITY **Field B** (complexity object): REAL_ALGEBRAIC_GEOMETRY

Statement:

For any AC^0 circuit C computing PARITY on n inputs, the communication complexity of the Karchmer-Wigderson game for C is $\Omega(\log n)$. This invariant $\psi(C)$ = communication complexity of C 's function is polynomial-time computable and invariant under input permutations.

Rationale (proposer's reasoning):

The Karchmer-Wigderson game links circuit size to communication complexity. By framing PARITY's complexity in real algebraic geometry (via polynomial representations of communication protocols), we isolate a structural invariant. This avoids natural proofs by focusing on non-trivial geometric obstructions to AC^0 computation.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7e8e2f324d671128

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n):
        # Generate a simple AC0 circuit for PARITY
        circuit = []
        for i in range(n):
            circuit.append((i, 'NOT'))
        return circuit

    def simulate_protocol(circuit):
        n = len(circuit)
        protocol = [(i, 1) if random.choice([0, 1]) == 0 else (i, -1) for i in range(n)]
        return protocol

    def compute_communication_complexity(protocol):
        return len(protocol[0])

    n = random.randint(5, 40)
    circuit = generate_ac0_circuit(n)
    protocol = simulate_protocol(circuit)
    communication_complexity = compute_communication_complexity(protocol)

    c = 1.0
    conjecture_holds = communication_complexity >= c * math.log(n)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "communication_complexity",
        "metric_value": communication_complexity,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
```

```

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

{'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 27, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 21, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 18, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 15, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 7, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 25, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 23, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 9, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 38, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 36, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 19, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 17, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
RESULT: SUPPORTED mean=20.766666666666666 std=9.03579302305866 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Single-instance testing cannot meet the 80% support threshold for statistical validity. |
 next: Test ≥ 1000 instances to validate support fraction and ensure statistical significance

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	98703
2	preregistration	ollama_remote	qwen3:8b	0	0	27752
3	novelty	ollama_remote	qwen3:8b	0	0	24028
4	novelty	ollama_remote	qwen3:8b	0	0	16258
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12314
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7271
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7587
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6865
9	critic	ollama_remote	qwen3:8b	0	0	33181
10	judge	ollama_remote	qwen3:8b	0	0	20684

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 254643 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b77a7e4e1eda.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b77a7e4e1eda.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b77a7e4e1eda.tar.gz` (if generated)